Open Access | https://doi.org/10.25080/tuvm4543



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

The Space to Build It: How Scientific Needs Shaped an Ecosystem

K. Jarrod Millman[1]

[1]Berkeley Institute for Data Science, UC Berkeley

Abstract

Open, community-owned scientific software does not emerge from good intentions alone. It grows from real scientific needs, cross-domain collaboration, community coordination, and people willing to bet on one another. This paper traces one path through that combination, told from a single vantage point among many who could tell it differently. My work in neuroimaging prompted me to co-organize a 2005 meeting at UC Berkeley that already brought together astronomers, neuroscientists, physicists, and statisticians working on problems of their own. From that meeting, my collaborations grew, over the following two decades, to include people across dozens of other fields, as more and more of us trusted one another to lead work outside our own specialties—the same bet that let me help grow NumPy, SciPy, and the community and conference around them, and that still shapes the scientific Python ecosystem today. The paper follows this shared path from its earliest concerns, through the architectural decisions and often invisible maintenance work that turned volunteer projects into shared infrastructure, and into the deliberate effort to recreate those early patterns at a larger scale. It closes with a reflection on who is trusted with the room and time to build such infrastructure, and who pays for that time when their institutions will not.

Keywords Scientific Python, NumPy, SciPy, Neuroimaging, Berkeley

1. Introduction

Open scientific software infrastructure often begins with a single actor—a researcher, a lab, a company—whose vision, energy, and effort make its creation possible. And, while that initial focused, isolated effort may be required to start, it matters greatly what happens next. Durable infrastructure is built in layers: a technical foundation, followed by community governance, and then, ideally, institutional support. Each layer changes who holds the infrastructure: first from a single owner to a community, and later from informal practice to institutional support. Each layer must be deliberately built, and rebuilt, as the infrastructure grows. When those layers get built, the infrastructure becomes durable. When they do not, growth stalls and the burden of keeping it alive rests on a shrinking circle of people.

This paper is personal. It is shaped by the collaborations I was part of and the problems that were in front of me at the time. Many of the people involved in this work would tell the story differently, from their own vantage points and concerns.

Each layer, once built, eventually stops working at the next scale, forcing the community to decide, consciously or not, whether and how to rebuild it. The three sections that follow trace that building and rebuilding in turn: how the technical foundation got built (“How to start”), how the community that depended on it grew past what that foundation alone could hold together (“How to grow”), and how institutional anchors began to emerge, unevenly, to sustain some of what community practice alone could not carry indefinitely (“How to last”). A single mechanism recurs across all three: aspirations become plans, and plans become foundations, only when a small, cross-disciplinary group shares a room, more than once,

Published Jul 13, 2026

Correspondence to
K. Jarrod Millman
millman@berkeley.edu

Open Access

Copyright © 2026 Millman. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

rather than negotiating asynchronously from a distance. But describing how open scientific software infrastructure started, grew, and lasted explains the conditions that made the work possible; it does not explain why researchers, scientists, and teachers who were not primarily programmers kept doing it, often unpaid and unrecognized by the institutions that employed them. That deeper question—who is trusted with the room and time to build such infrastructure, and who pays for that time when their institutions will not—is the one this paper answers last, because the answer only becomes visible once the full three decades of scientific Python are in view.

2. How to start

In 2003, I supported a neuroimaging analysis pipeline at UC Berkeley that was an *ad hoc* collection of MATLAB toolboxes, IDL routines, C/C++ programs, AWK scripts, and shell glue that nobody fully understood. Results from our center and others were difficult to verify. There was no mechanism to rerun an analysis from raw data. Scientific integrity was a clear concern, and it was made worse by technical constraints. The academic incentive system amplified the problem: writing and maintaining analysis software did not appear in tenure files, open source authorship diluted traditional credit models, and computational reproducibility was not a recognized research output.

My colleague, Matthew Brett, was initially concerned less about reproducibility itself and more about the way neuroimaging software concentrated power: the dominant analysis tools were effectively single-owner open source, with one lab setting priorities and methods for the field. If you disagreed with those methods, your options were to fork that single-owner project or to invest in a community-owned alternative that could be governed by the wider neuroimaging community. That made the politics of the software visible in a way I had not thought much about before.

2.1. A risky solution

Perhaps naively, Matthew and I set out to build a community-owned neuroimaging platform in Python that would make analyses scriptable from raw data to published results [1]. Python looked like the right language, but the surrounding platform was still only half-formed. Travis Oliphant, Eric Jones, and Pearu Peterson had released `scipy` by combining packages that they each wrote as a "superpackage" of numerical integration, optimization, special functions, and more [2]. It was built on top of `numeric`, a multi-dimensional array package maintained by Paul Dubois at Lawrence Livermore National Laboratory. Jonathan Taylor had already begun `brainstat` for functional magnetic resonance imaging (fMRI) analysis, John Hunter (`matplotlib`) was developing `pbrain` for electroencephalogram (EEG) and electrocorticography (ECoG) analysis, and John introduced us to Fernando Pérez (`ipython` and later `jupyter`), connecting our neuroimaging effort to the broader Python tools that were taking shape around it.

At the same time, Perry Greenfield's team at the Space Telescope Science Institute had introduced `numarray`: a second, incompatible array implementation aimed at very large images. New projects were encouraged to use it, older ones could not easily migrate, and no agreed path to unification existed [3], [4]. Meanwhile, the `numpy-discussion` list provided a shared communication space across this divide, so the same people were discussing `numeric` and `numarray` even as their code bases remained split.

2.2. The 2005 meeting

As we planned a neuroimaging meeting for March 2005 at Berkeley, we were watching the `numpy-discussion` threads that followed Travis's January `numeric3` design proposal [5] and February status update [6]. In those posts, Travis described `numeric3` as a bridge between

`numeric` and `numarray`, based largely on the `numeric` code base, and emphasized the need to keep `numeric`'s tight homogeneous-array implementation moving forward while still learning from `numarray`. The idea that `numeric` and `numarray` should eventually merge was there, but only as an aspiration. By inviting Travis (`numeric3`) and Perry (`numarray`) to join the Berkeley meeting and to speak with Guido van Rossum (`python`) and Paul (`numeric`), we hoped to turn that aspiration into a shared plan for a unified array object and package.

The first day was a hands-on laboratory, "*Scientific Python for Neuroscience Research*," taught by Fernando and John and designed to make `numeric`, `scipy`, `ipython`, `matplotlib`, and related tools immediately usable for working neuroimaging scientists. The next two days were split into two parallel tracks: one, focused on neuroimaging; the other worked through whether and how a unified array object could be built that preserved the strengths of both `numeric` and `numarray`. Each evening we reconvened to discuss the day's work.

After the meeting, two `numpy-discussion` threads communicated the outcomes and made the unification plan explicit. Perry's "*Notes from meeting with Guido regarding inclusion of array package in Python core*" [7] concluded that including an array package in Python would be a distraction that would sap energy from the unification work that needed to happen. Travis' "*Future directions for SciPy in light of meeting at Berkeley*" [8] opened a public discussion that quickly resolved into a modular plan for SciPy: a minimal, easy-to-install `scipy_core` that could replace both `numeric` and `numarray`; `scipy` itself as a separate package (or series of packages) of algorithms built on that core; plotting moved out into its own libraries such as `matplotlib`; and domain-specific packages, for astronomy or neuroimaging, living as independent projects rather than inside `scipy`. Travis committed to spending the following five to six months making `scipy_core`. That commitment culminated, eighteen months later, in the release of `numpy 1.0` in October 2006 [9], [10].

What happened in Berkeley reached further than the emails alone suggest. On the long drive to and from lunch with Guido and Paul, Travis and Perry went over numarray's internals in detail, and several of its solutions to shared problems, including a memory-efficient approach to type casting of arrays, found their way into what became `numpy`. The eventual library kept numeric's efficiency for small arrays, but its user interface and many higher-level features were closer to numarray than to Numeric.

2.3. Building the foundation

While transformational in its unification of the array interface, `numpy 1.0` was just the beginning of what needed to be built. A scientific computing platform, comparable to what MATLAB or IDL provided, required far more: a vast collection of mature, performant algorithms spanning optimization, signal processing, linear algebra, statistics, differential equations, and more; a plotting system; an interactive environment; and the documentation, testing, and packaging infrastructure that makes all of it installable, trustworthy, and teachable.

In December 2007, I organized a Berkeley sprint with Travis Oliphant, Eric Jones, Robert Kern, and others, with newer contributors like David Cournapeau and Stéfan van der Walt participating remotely, to put together a `scipy 1.0` roadmap, improve the organizational structure around `scipy`, and plan more activities to help `scipy` development build momentum. After the sprint, Travis' `scipy-dev` email communicated the outcomes and announced a new `scipy` board (Jones, Kern, Millman, Oliphant) with a specific commitment: monthly virtual doc-days and bug-days, coordinated over IRC, with at least one board member present at each [11]. Over the following two years, David and Stéfan became two of the most active committers to both `numpy` and `scipy`, working alongside long-time contributors, remote collaborators, and newer volunteers to restructure and expand the library, build and improve the test infrastructure, standardize and vastly expand the documentation, and

create development processes and tooling that made contributions from a widely distributed, cross-disciplinary community sustainable. By the time `scipy 0.7` shipped in February 2009, the platform that had been rough and fragile in 2007 was something scientists could install, trust, and build on [12], [13], [14].

2.4. Lesson 1

What started as a neuroimaging software problem did not stay one. Looking for a way to build community-owned neuroimaging tools, rather than relying on single-owner lab software, pulled me and my collaborators into the broader scientific Python effort, because the array foundation that any such shared platform would need was itself unresolved. Solving our problem meant first helping solve `scipy`'s.

The 2005 Berkeley meeting turned out to be decisive. What made it work was not the agenda but the room: a hands-on lab that got working scientists using the tools immediately, two parallel tracks that let the neuroimaging discussion and the array negotiation proceed side by side, and evenings when both groups came back together to compare notes. Travis and Perry were in the building, not posting to a list—and the difference showed in what the discussion produced.

What it produced was a plan: a unified array object, a roadmap that ran through `numpy 1.0` and, two years later, the 2007 sprint's board. The people in that room (neuroimagers, astronomers, statisticians) were oriented around one shared technical problem: having a foundation solid enough to build on. None of us was yet thinking about how to develop, or hold together, a distributed collection of projects across dozens of scientific domains.

> **Shared problems need shared rooms.**
>
> A mailing list can carry information, but it cannot carry the trust required to trade a position for a compromise; that trade happens face to face, and only when the people trading it have been given the standing, and the time, to spend on it. The rooms get bigger later (a sprint, then a summit), but the trade being made in each one is the same one this paper returns to at the end.

3. How to grow

For the first decade, `numpy` and `scipy` developers were essentially the same people, releasing two different packages from the same small, tightly connected group. The `scipy` toolkits, or `scikits`, began as another layer that we built on the same shared infrastructure, largely with the same hands. The SciPy conference was organized by that same small group as well. Over the decade that followed, each of these developed a community of its own, increasingly separate from the others and, eventually, separate from the people who had founded them.

3.1. Infrastructure and diaspora

In February 2009, Stéfan van der Walt posted to the `scipy-dev` list with a subject line that, unknown to him, echoed Travis Oliphant's email after the 2005 Berkeley meeting: "*The future of SciPy and its development infrastructure*" [15]. In Travis' 2005 email, he had worried that there were "few SciPy devotees" and even fewer contributors. Now, four years later, Stéfan worried that `scipy` was a "big library of code, used by many scientists," but still maintained by only a few people.

Stéfan's diagnosis pointed to a different kind of crisis: getting commit access was too hard, and contributors' patches could sit for a year or more without feedback. His proposed solution was distributed version control, formal code review, and higher standards for tests

and documentation. There was no community consensus about distributed version control at the time; for some of us it looked like necessary infrastructure, for others like unfamiliar machinery that might raise the barrier to contribution rather than lower it. Travis worried that adding formal process would drive away contributors. Stéfan and David Cournapeau argued that without it, the burden would become unsustainable.

Gaël Varoquaux, a member of the neuroimaging team, wrote to me the Saturday after SciPy 2009 [16]. He had just run a Birds of a Feather session on establishing a standard machine learning package, building on the `scikits.learn` code from David Cournapeau's 2007 GSoC project that I had mentored. The enthusiasm had been "huge," he wrote, and he was ready to move quickly. But "in order to streamline the process, we need a mailing list, version control and a website."

Getting a new project onto the SciPy infrastructure meant asking a busy administrator, sometimes including me, to add it. The server itself was failing: `httpd` crashed spontaneously and had to be restarted manually, and adding new projects was difficult and time-consuming. The infrastructure was not failing because anyone was neglectful. It was old hardware, a layered history of configuration decisions, and an operating system that needed a full reinstall, all sitting under a fast-growing collection of libraries and a small group who could no longer maintain it on the side.

Given the server situation, I recommended that Gaël look into other options. That conversation was not unique; projects from `statsmodels` to `scikit-image` were reaching the same conclusion by the same route. The `scikits` had started as extensions to `scipy`, built by much the same small group, on the same shared infrastructure. Once they left that shared infrastructure, most stopped being viewed as scipy extensions and increasingly grew into independent software projects with their own developer communities. `scikits.learn` became `scikit-learn`; other projects grew up with no connection to the `scikits` name or idea whatsoever. What had been imagined as one layer of a tool stack (`numpy`, `scipy`, and the `scikits`) became part of a larger *computational ecosystem*: a loose collection of interoperating Python tools and communities, organized around shared foundations but developed, governed, and sustained by independent projects [17], [18].

3.2. Community and conference

For the first six years, Enthought organized the conference, while Caltech's Center for Advanced Computing Research hosted it; Enthought also sponsored students and handled logistics. In 2008, Travis Vaught of Enthought and I served as co-chairs, and Gaël Varoquaux served as program chair. Together, we recruited tutorial coordinators, webmasters, and a program committee. For the first time, a proceedings review process produced published conference proceedings, reviewed in the open by identifiable proceedings reviewers [19]. By creating community groups responsible for tutorials, the program, and other conference decisions, we shifted responsibility from Enthought to a broader community that could decide what the conference would be. That year also saw the first EuroSciPy, held in Leipzig, Germany. In 2009, Prabhu Ramachandran and I co-chaired the first SciPy India Conference.

Conference proceedings, published and citable, were a mechanism for converting the work done on creating and maintaining software libraries into something a tenure file could recognize. Papers were submitted as reStructuredText source in a public GitHub repository; review happened in the open, via pull requests attached to identifiable individuals; reviewers were acknowledged by name. The toolchain, developed with Gaël Varoquaux and extended by Stéfan van der Walt into a system using Sphinx, LaTeX, custom scripts, and the `procbuild` preview bot, was an early instance of what would later be called "open peer review" and put into practice the principles I articulated in a 2012 paper titled "*Learning from Open Source Software Projects to Improve Scientific Review*" [20]. Its goal was explicitly

iterative: reviewers and editors worked with authors to guide papers toward acceptance, so that submitting to the SciPy conference proceedings meant opening your writing to the same collaborative scrutiny that good open source development demands.

My last year as co-chair for the SciPy conference was 2011. The people organizing the conference by then were no longer, by and large, the people maintaining the library. That shift was deliberate in a way the infrastructure diaspora was not. The `scikits` left `scipy.org` because the server and processes could no longer scale; the conference moved because we wanted it to—from company stewardship to community governance, from one annual meeting in California to a network of meetings on three continents. Proceedings made it easier for academic developers and users of scientific Python to justify attending; as more of those people came, the organizers naturally became drawn from a broader pool. By the end of the decade, "SciPy the conference" was clearly its own institution, with its own leadership and rhythms, loosely connected to the library and no longer run by the same small group.

3.3. Lesson 2

A shared technical foundation is more than scaffolding: it is part of what holds a community together. But holding together and staying uniform are not the same thing. When that foundation stopped scaling, the people who depended on it did not disappear; they found their own paths, and what followed looked, from the outside, like fragmentation. From the inside it was something closer to growth: the same people, carrying norms first worked out together, building new things in new places, and discovering by experiment what a much larger ecosystem would need next.

The conference took a different route to the same outcome. Rather than scattering under pressure, it was deliberately opened: from a company-run event serving one small circle to a community-governed institution with its own proceedings, its own international meetings, and eventually its own leadership drawn from far beyond the group that had built `numpy` and `scipy`. The infrastructure diaspora and the `scikits`' departure were reactions to a shared foundation that no longer scaled. The conference followed another path: as it grew, more participants could take responsibility for its program, tutorials, proceedings, and meetings.

> **Growth changes what needs to be held together.**
>
> As the `scikits` and the conference grew, the arrangements that once joined them to `scipy` no longer fit what each needed. The lesson is to recognize those transitions and deliberately develop the infrastructure, roles, and governance that a larger community requires.

4. How to last

At the end of the second decade we faced a new problem: there was no longer anyone with the mandate, or the mechanism, to hold the ecosystem together. As individual projects matured, scientific Python communities serving particular scientific domains (e.g., neuroimaging, geospatial data science, and high-energy physics) developed their own coordination structures, mailing lists, and governance conventions. This domain-level organization was healthy in many respects. It also meant that an increasing amount of work that was really ecosystem-wide in scope, packaging standards, CI practices, documentation tooling, was being developed in domain-specific silos and reinvented independently across communities. Newer maintainers and contributors built careers within a single project or domain rather than across several. The cross-disciplinary experience that those of us who had been there at the beginning carried was no longer naturally transmitted to the next generation. The `scipy.org` website (nominally maintained by the `scipy` library team,

effectively serving as the ecosystem's front door) had become a symbol of that failure of transmission: a shared resource that no one had the mandate to govern.

Some of those coordination needs had already become visible. A group of contributors from across scientific Python proposed a set of recommendations that would apply across projects, but used NumPy's proposal process because no ecosystem-wide forum yet existed. It was a useful solution, but it also made the underlying problem visible: standards intended for many independent projects could not depend indefinitely on any one project's processes or governance.

4.1. Ecosystem coordination

In 2020, Stéfan and I co-founded the *Scientific Python* project with support from the Alfred P. Sloan Foundation [21] to provide what the ecosystem needed, not the SciPy server's centralized infrastructure, which individual projects had rightly outgrown, but a shared layer above the individual projects: cross-project recommendations, developer summits that worked like the early sprints (small, cross-disciplinary, and work-focused), shared tooling and discussion forums, and a common landing page for the ecosystem `scientific-python.org`. Early community outreach by Juanita Gomez and project leadership by Brigitta Sipőcz were especially important in turning *Scientific Python* from a small node into a functioning ecosystem hub.

The 2023 Scientific Python Developer Summit illustrated what this cross-project coordination makes possible. Henry Schreiner had developed a comprehensive development guide for the `scikit-hep` community, high-energy physics, that covered modern packaging, testing, CI, documentation, and more; in parallel, Dan Allan at the National Synchrotron Light Source II had developed complementary guidelines for his community. Both guides addressed concerns that had nothing to do with the specific scientific domain and everything to do with the shared challenge of building maintainable scientific Python packages. At the summit, that work was merged, rebranded as an ecosystem-wide resource, and released as the Scientific Python Development Guide [22], [23]: a signal, concrete and deliberate, that domain-specific silos could be dissolved when the right forum existed to bring people together. The summit worked because it worked like the early meetings and sprints: small, cross-disciplinary, organized around real work rather than reports.

4.2. Academic anchoring

The story so far has been mostly about software: an array object, a library, an ecosystem, a software conference, a coordination layer. But this ecosystem changed more than code: it changed the people and the places that built it. Our neuroimaging collaboration brought Fernando Pérez to Berkeley's Brain Imaging Center (BIC) as a research scientist around 2008, where he continued developing IPython and, not long after, co-founded Project Jupyter. Over the years that followed, scientific Python and Jupyter colleagues who came to Berkeley for a meeting, a sprint, or a few months of collaboration became part of life at the BIC and, later, at Berkeley more broadly. Min Ragan-Kelley (`ipython` and later `jupyter`) was among those who first came to work alongside us this way, one of many such visits I won't try to enumerate. The pull reached graduate students, too. Kirstie Whitaker, then a neuroimaging PhD student, has described how Cindee Madison (part of our neuroimaging collaboration) mentored her in learning Python and navigating open source norms and culture, ultimately reshaping her career. Kirstie's is one version of a story that played out, in smaller ways, across generations of Berkeley researchers who found themselves drawn into the ecosystem they had only meant to borrow a tool from.

That gravitational pull eventually helped reorganize the university around it. The same community of scientific Python and Jupyter contributors that had been informally passing

through Berkeley helped establish the Berkeley Institute for Data Science (BIDS), which Fernando co-founded, and BIDS' early success fed into the university's decision to launch the College of Computing, Data Science, and Society and, within it, a Data Science major that is now the largest on campus [24]. The undergraduate curriculum built on that foundation, Data 8 and Data 100, uses NumPy, SciPy, Matplotlib, and Jupyter notebooks as the substrate for the largest introductory and largest upper-division courses at Berkeley, where thousands of students each semester use computation as part of learning statistics and data science [25]. That curriculum did not stay on campus: more than fifty universities and colleges have since adopted Data 8 or its components, several through Berkeley-supported partnerships with the National Science Foundation's CloudBank project that extended the same open-source, cloud-based teaching stack to community colleges and other UC campuses [26].

Fernando is now the BIDS' Faculty Director, a decade after co-founding it; Stéfan and Min work there on the technical and infrastructure side; Kirstie, the graduate student whose career had once been redirected by conversations with Cindee, returned in 2025 as BIDS's Executive Director; and I lead Berkeley's Open Source Program Office from within it. Fernando has written that Berkeley's leadership in this stack "is not 'owned' by Berkeley," but built by scientists who "partner with an extended, distributed community of other researchers and developers to build an ecosystem that benefits all" [27]. This is the same argument this paper opened with in 2003—that community-owned tools, built and governed by the people who need them rather than by a single lab or company, outlast anything one owner can sustain alone—now stated at the level of a university rather than a single lab.

What comes next is not yet settled. Fernando has described BIDS as a space for open scholarship, open source, and interdisciplinary collaboration on AI in science and society [27], and how artificial intelligence reshapes the way scientists write code, teach it, and learn it is the open problem this community will now have to work out together, in much the same way it once worked out the future of the array object in a room in Berkeley.

4.3. Lesson 3

Why did two of the most widely-used projects in scientific computing (the scientific Python and Jupyter ecosystems) have roots inside a neuroimaging lab at Berkeley that was not, on paper, in the software business at all?

One condition that mattered enormously was how Mark D'Esposito ran his lab. Mark wanted good people in the lab producing good work, and he trusted his team enough to let them pursue what they judged important, even when it sometimes drifted far from neuroscience. That trust is what let Matthew, Fernando Pérez, and me spend real time on scientific Python and, later, Jupyter, work that was not in any of our job descriptions and that did not, on its own, advance the neuroscience questions the lab was trying to answer.

Mark gave me the freedom to decide, together with Matthew and Fernando, what the major problems on the horizon were, and then to work on them. He gave us shared space, in the literal sense of desks in the same building, and a subtler kind of space: freedom from having to justify that work in terms of our immediate career prospects. That combination, more than any specific decision about software or method, is what let a small group inside a neuroimaging lab become a nucleus that drew in others and helped grow two community-owned ecosystems.

Being embedded in a neuroimaging lab rather than a software lab shaped our work. Our colleagues were neuroscientists, not programmers, so anything we built had to be installable, learnable, and usable by people who had no interest in becoming software

developers. That constraint pushed us toward teaching as much as toward building: writing documentation and tutorials, running hands-on sessions, and treating the question of how someone would learn a tool as inseparable from the question of how to design it.

Our collaborations with astronomers, physicists, and statisticians meant our tools were constantly tested against needs that had nothing to do with neuroimaging, which kept our discussions from narrowing into a closed conversation among people who already agreed with each other. It also connected us to larger conversations about open science and reproducibility that were happening well outside any one department, so the problems we were facing were visibly the same problems other fields were running into.

Matthew, Fernando, and I were lucky. Many of the people who built and sustained this ecosystem did not have a Mark D'Esposito, and did the work anyway: on nights and weekends, on volunteered time, on the strength of believing in something their institutions gave them no formal credit for. Some took real career risks to keep contributing. Many could not sustain that position indefinitely and left for industry, where the demands of a job rarely leave room for the kind of unstructured, long-horizon tinkering that built the scientific Python and Jupyter ecosystems. The space that made this possible was never distributed evenly, and a great deal of this ecosystem was built by people paying, personally, for a trust their own institutions were unwilling to extend.

BIDS is one attempt to make institutionally what Mark gave informally: room, trust, and time, backed by something more durable than one director's goodwill. But these remain exceptions rather than the norm, and they are not yet enough to catch everyone this ecosystem depended on and then let go.

It was the space to build it.

This ecosystem lasted because some people were trusted to decide what mattered and given time to work on it. It has also lost contributors, and capacity, because that trust was never extended widely or durably enough. Making it durable is the work still ahead.

5. Conclusion

This account is mine, but the pattern it traces is not. A concrete problem that outgrows its original domain, a small group that chooses to share a room until an aspiration becomes a plan, and a foundation later reorganized (deliberately or under pressure) into something many hands can sustain: this arc is not unique to numpy, scipy, or Berkeley, and other ground-up collaborations may recognize some version of it in their own histories. It is fitting that this account spans almost exactly the twenty-five years the SciPy conference itself has now completed. The same needs that pulled a handful of neuroimagers, astronomers, and statisticians into one room in 2005 are what still pull new communities into this one today, and the work of extending enough trust, time, and institutional support to sustain them is, a quarter century on, still not finished.

Acknowledgements

I am grateful to Matthew Brett, David Cournapeau, Perry Greenfield, Madicken Munk, Travis Oliphant, Pushkar Sathe, Stéfan van der Walt, and Kirstie Whitaker for their careful reading of the manuscript and for thoughtful comments that helped clarify the narrative and sharpen several of its central arguments.

Portions of this work were assisted using generative AI tools. Specifically, I used Perplexity AI for research, fact-checking, refining language, restructuring the paper, copy editing, and proof-reading. All outputs were reviewed, verified, and revised by me, and I take full responsibility for the accuracy and integrity of the final content.

References

[1] K. J. Millman and M. Brett, "Analysis of Functional Magnetic Resonance Imaging in Python," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 52–55, 2007, doi: 10.1109/mcse.2007.46.

[2] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, Feb. 2020, doi: 10.1038/s41592-019-0686-2.

[3] T. Oliphant, "Status of Numeric." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2004-January/002630.html

[4] T. Oliphant, "Comments on the Numarray/Numeric disscussion." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2004-January/002667.html

[5] T. Oliphant, "updating Numeric." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2005-January/003906.html

[6] T. Oliphant, "Numeric3 status." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2005-February/003958.html

[7] P. Greenfield, "Notes from meeting with Guido regarding inclusion of array package in Python core." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2005-March/004354.html

[8] T. Oliphant, "Future directions for SciPy in light of meeting at Berkeley." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2005-March/004318.html

[9] T. Oliphant, "[ANN] NumPy 1.0 release." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2006-October/011810.html

[10] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[11] T. Oliphant, "SciPy Sprint results." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2007-December/030263.html

[12] K. J. Millman, "ANN: SciPy 0.7.0." [Online]. Available: https://mail.python.org/pipermail/scipy-dev/2009-February/010977.html

[13] S. Developers, "SciPy 0.7.0 Release Notes." [Online]. Available: https://docs.scipy.org/doc/scipy/release/0.7.0-notes.html

[14] K. J. Millman and T. Vaught, "The State of SciPy," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 2008. doi: 10.25080/HNCD8803.

[15] S. van der Walt, "The future of SciPy and its development infrastructure." [Online]. Available: https://mail.python.org/pipermail/scipy-dev/2009-February/011109.html

[16] G. Varoquaux, Personal communication, 2009.

[17] K. J. Millman and M. Aivazis, "Python for Scientists and Engineers," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 9–12, 2011, doi: 10.1109/MCSE.2011.36.

[18] F. Pérez, B. E. Granger, and J. D. Hunter, "Python: An Ecosystem for Scientific Computing," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 13–21, 2011, doi: 10.1109/MCSE.2010.119.

[19] G. Varoquaux, T. Vaught, and K. J. Millman, "Editorial," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 2008. doi: 10.25080/HXDH2061.

[20] S. S. Ghosh, A. Klein, B. Avants, and K. J. Millman, "Learning from open source software projects to improve scientific review," *Frontiers in Computational Neuroscience*, 2012, doi: 10.3389/fncom.2012.00018.

[21] S. van der Walt and K. J. Millman, "Scientific Python: Planning the Next Decade," Grant Proposal, 2020. [Online]. Available: https://scientific-python.org/doc/scientific-python-planning-grant-2020.pdf

[22] H. Schreiner, "The Scientific Python Development Guide." [Online]. Available: https://blog.scientific-python.org/scientific-python/dev-summit-1-development-guide/

[23] Scientific Python Community, "Scientific Python Library Development Guide." [Online]. Available: https://learn.scientific-python.org/development/

[24] Brandilyn Buckley, "UC Berkeley Data Science major now largest on campus." [Online]. Available: https://cdss.berkeley.edu/dsus/news/data-science-major-now-largest-campus

[25] E. V. Dusen, A. Suen, A. Liang, and A. Bhatnagar, "Accelerating the Advancement of Data Science Education," *SciPy 2019*, 2019, doi: 10.25080/Majora-7ddc1dd1-000.

[26] CloudBank, “CloudBank for Educators.” [Online]. Available: https://www.cloudbank.org/training/cloudbank-educators

[27] F. Pérez, “Director's Vision 2024.” [Online]. Available: https://bids.berkeley.edu/about/directors-vision-2024